

Conv1

Assumptions

Input: 8 8 1

- greyscale image
- each pixel will be 8-bit int

Kernel: 3 * 3

- Stride -> 2
- Padding -> 1 (to make sure i get all the border values and don't lose data)
- 1 channel -> greyscale image
- 9 bytes of weights -> will be stored in the cache(local memory) -> Weight Stationary Dataflow

Constraints:

- 4 MACs -> each 8-bit * 8-bit -> 16-bit accumulate & output -> overflow policy - round/clip (assumption)
- 16 bytes local memory -> SRAM (cache memory assumption)
- 1 KB/s external memory transfer bandwidth -> 1024 B/s
- Inputs and outputs will be streamed

Calculations

Cycles and patch

Number of MAC operations -> $16 \cdot 9 = 144$

number of MAC -> 4

Cycles -> $144/4 = 36$ cycles + 32 cycles -> 68 cycles (theoretical)

for each output 9 MAC operations,

pairwise reduction with 2 MACs will be used to complete the partial sum calculations and complete the operation -> this will take 2 cycles

$2 \cdot 16 = 32$ cycles

While reading later i found that i had missed the final accumulate stage in these handdrawn calculations

therefore MAC cycle frequency is $68 \cdot 12.8 = 870.4 = 871$ Hz (approx)

Cycle 0: compute patches 0 to 3

Cycle 1: compute patches 4 to 7

Cycle 2: compute patch 8

Cycle 3: $MAC0 \leftarrow ACC0 + ACC1$ $MAC1 \leftarrow ACC2 + ACC3$ pairwise adds

Cycle 4: $MAC0 \leftarrow MAC0 + MAC1$ final add $\rightarrow$ final_sum

The ideal lower bound needs better packing, tighter control each cycle, and some changes to the internal hardware. With my current set-up, scheduling and control, I got a 5-cycle sequence per patch, so it ends up being $5 \times 16 = 80$ cycles per image. With the external bandwidth giving 12.8 images/s, the required MAC clock becomes $80 \times 12.8 \approx 1024$ Hz.

Stage	Action
Load weights $\rightarrow$ SRAM	One-time: read 3×3 weights (9 B) from external memory into 16 B SRAM.
Stream pixel window	Per patch: stream the 3×3 activation window from input FIFO onto the input bus.
MAC compute	3 cycles: multiplies across operands 0–3, 4–7, then operand 8 (others idle).
Serial reduction	2 cycles: pairwise add ($ACC0 + ACC1$, $ACC2 + ACC3$) $\rightarrow$ final add (sum of pairs).

Input = $8 \times 8 \times 1$ kernel = 3×3 stride = 2
Output = 4×4 . Padding = 1

$\rightarrow 16$ # of patches.
for each output = 9 MACs

Total operations = $9 \times 16 = 144$.

4 MACs, 16 byte SRAM, 1 kB/s external

For each input = 8 bits int.
only 1 channel.

16-bit O/P.

using weight-stationary $\rightarrow 9$ B
 $\therefore 7$ B left of 16.

$$O/P = \frac{(8 - 3 + (2 \times 1))}{2 + 1} = 4 \quad \therefore 4 \times 4 = 16 \text{ pixels}$$

Cycles needed

assuming each MAC takes 1 cycle & there are 4
 $144 / 4 = 36$ cycles per image.

Memory and transfer speeds

input $\rightarrow 8 \times 8 \times 1 = 64$ bytes $\rightarrow$ read

output $\rightarrow 4 \times 4 = 16$ bytes $\rightarrow$ write

total is = 80 bytes

weights $\rightarrow 3 \times 3 = 9$ bytes $\rightarrow$ read once and stored in SRAM

bandwidth $\rightarrow 1024 / 80 = 12.8$ images/s

Memory

I/P = 64 B. $\rightarrow$ read + 1.9 B read.
O/P = 16 B $\rightarrow$ write

bandwidth

~~1024~~ 1024 180 = 12.8 images

$\therefore$

MAC runs at $36 \times 12.8 = 460.8 \approx 461 \text{ kHz}$

Padding and stride

There will be 16 patches to ensure all data points are covered

$8 \times 8 \times 1$ → grayscale.

weights → 8 bits
kernel → $3 \times 3 \rightarrow 9$

stride = 1

$P = 0$

2 overlap

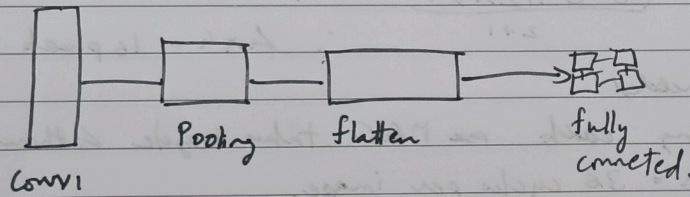
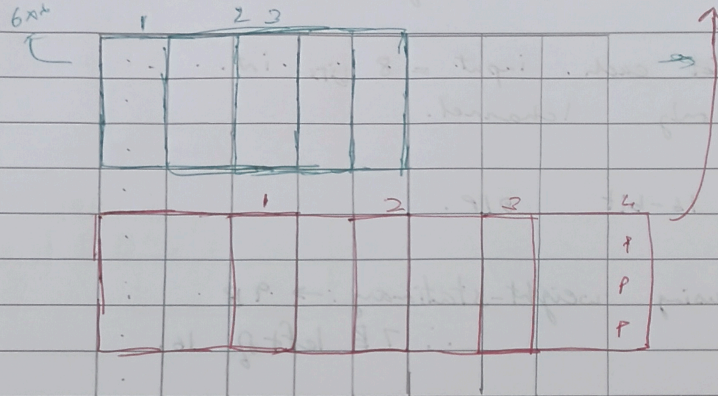
↓
 6×6

Stride = 2

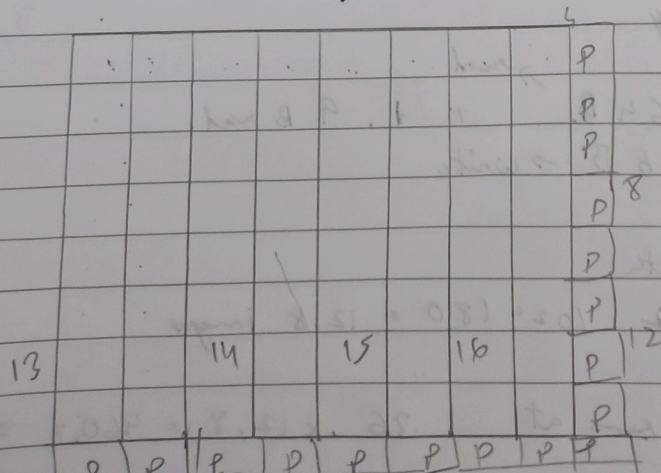
$P = 1$?, $P = 0$

1 overlap.

↓
 4×4



cells that need padding → 4, 8, 12, 13, 14, 15, 16.

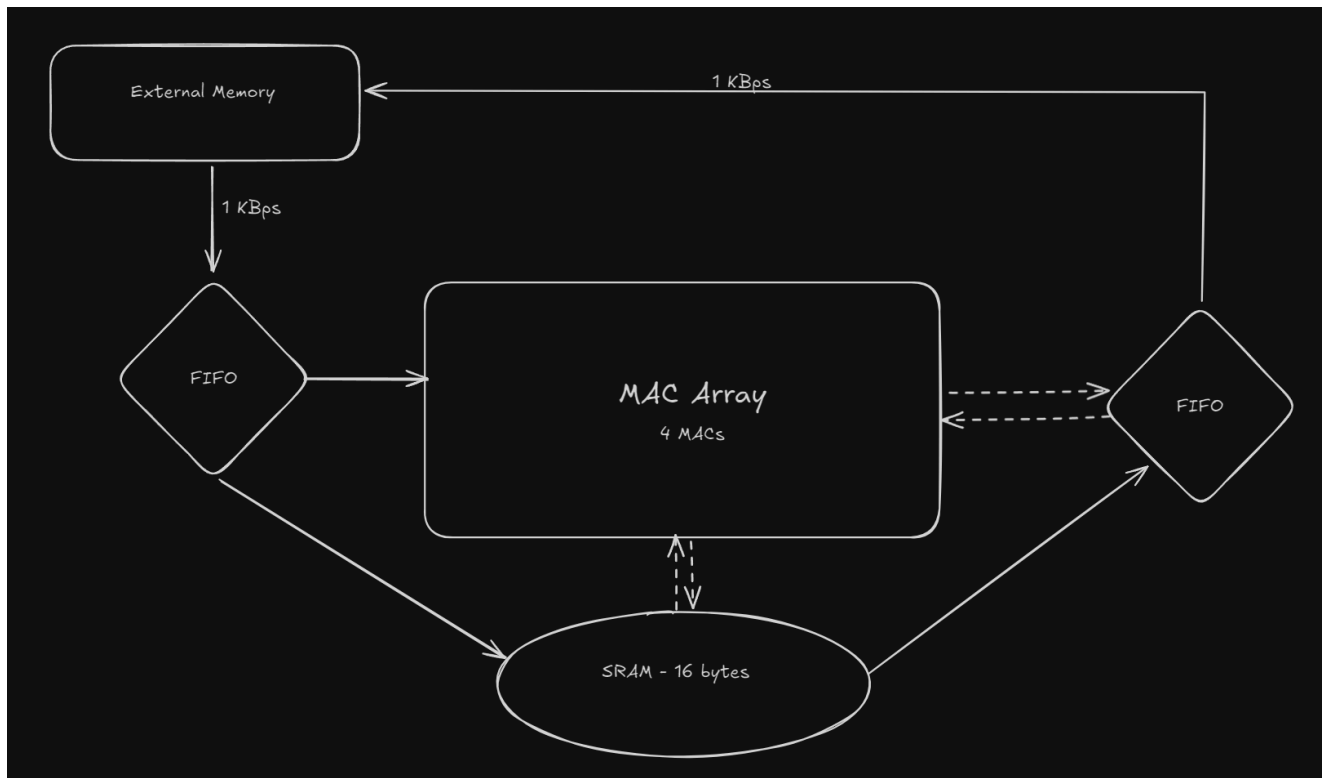


Architecture

System Overview:

The below figure shows the Conv1 system, where the main constraint is the tiny 16-byte on-chip SRAM. This memory acts only as a scratchpad and permanently stores the 3×3 (9-byte)

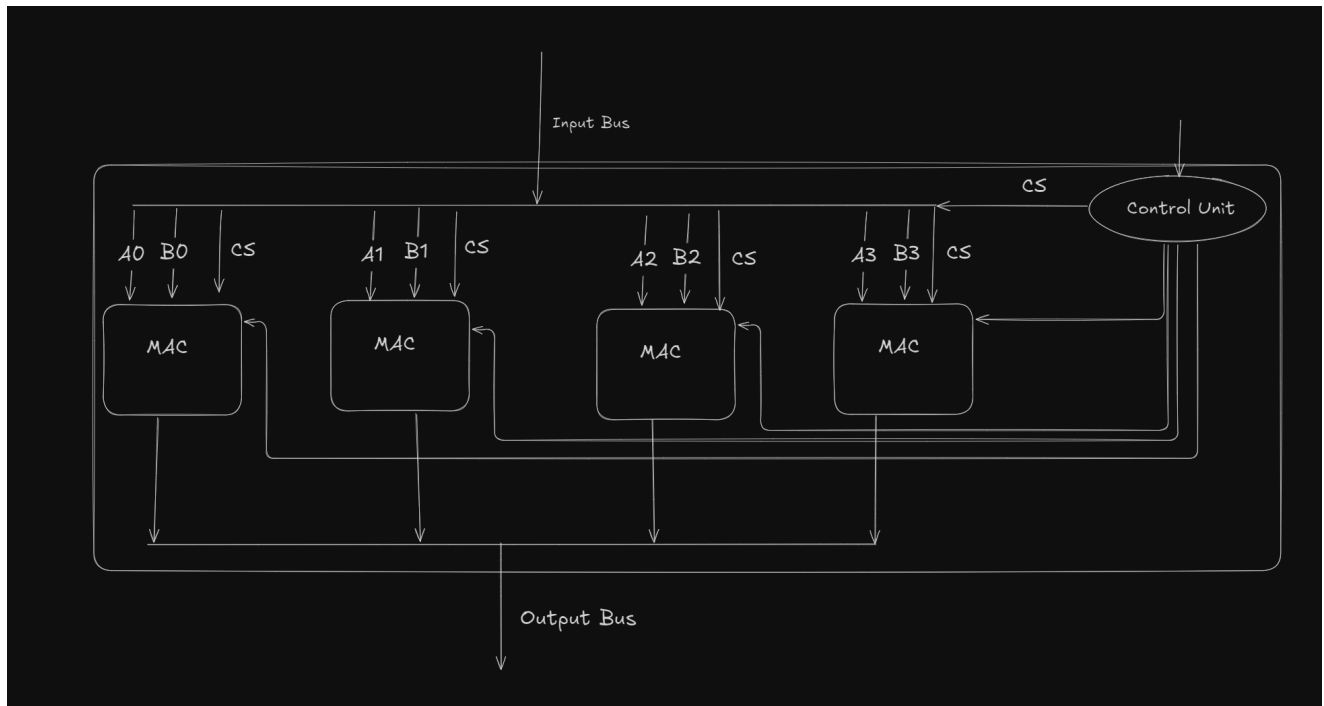
filter weights, leaving almost no space for activations or patch buffering. As a result, all input pixels must stream directly through the MAC array, making the design weight-stationary and fully streaming. The input and output FIFOs form the real synchronization layer, smoothing the flow between external memory and the compute core. The diagram reflects this lightweight loop: external memory → FIFOs → MAC array → FIFOs → external memory.



MAC Array structure:

The below figure shows the organization of the four-lane MAC array and its associated control plane. The array operates in a SIMD-style configuration, where a single Control Unit broadcasts the relevant timing and operation signals to all MAC units simultaneously. An input bus delivers pixel values to all lanes in parallel, enabling concurrent multiply-accumulate operations across the four MACs. The dedicated CS (control-signal) lines shown in the figure carry cycle-accurate commands such as register write-enable, mux-select, and accumulation triggers. These signals allow the Control Unit to coordinate the serial-reduction phase, where specific MACs are selectively activated to write computed partial sums back to

the shared SRAM in a controlled sequence.



Cycle	MAC0	MAC1	MAC2	MAC3	Notes
0	mult $a_0 \cdot w_0$	mult $a_1 \cdot w_1$	mult $a_2 \cdot w_2$	mult $a_3 \cdot w_3$	compute operands 0–3
1	mult $a_4 \cdot w_4$	mult $a_5 \cdot w_5$	mult $a_6 \cdot w_6$	mult $a_7 \cdot w_7$	compute operands 4–7
2	mult $a_8 \cdot w_8$	idle	idle	idle	last operand; other MACs idle
3	$ACC0 \leftarrow ACC0 + ACC1$	$ACC1 \leftarrow ACC2 + ACC3$	idle	idle	pairwise reduction
4	$ACC0 \leftarrow ACC0 + ACC1$ (final)	idle	idle	idle	final accumulate $\rightarrow$ final output ready

MAC Internal View

The below figure illustrates the internal micro-architecture of a single MAC unit. Each MAC contains input registers feeding a combinational multiplier and adder, with the results captured in a dedicated accumulative register. A 2:1 multiplexer in the feedback loop allows the Control Unit to toggle between two operating modes: accumulation mode, where partial sums continue building over several cycles, and reset mode, where the accumulator is cleared without incurring additional compute overhead. This structure supports an output-stationary flow, meaning that partial sums remain local within the MAC until the reduction phase completes. The arrangement minimizes data movement and avoids unnecessary

SRAM traffic while keeping the reduction logic straightforward.

